Chapter 2
Evasion Attacks

PCC190 – Secure Machine Learning
Prof. Rodrigo C. P. Silva – DECOM/UFOP

October 25, 2026

Contents

Setting and notation **2**

1 Definitions: Evasion Attacks **2**
1.1 Informal Definition . . . 2
1.2 Formal Definition . . . 2

2 A Taxonomy of Evasion Attacks **3**

3 White-Box Attacks **4**
3.1 Budget problem: FGSM, BIM and PGD . . . 4
3.1.1 Fast Gradient Sign Method (FGSM) . . . 5
3.1.2 Basic Iterative Method (BIM) . . . 6
3.1.3 Projected Gradient Descent (PGD) . . . 6
3.2 Min-norm problem: Carlini and Wagner, and DeepFool . . . 8
3.2.1 Carlini and Wagner . . . 8
3.2.2 DeepFool . . . 9

4 Query-Based Black-Box Attacks **10**
4.1 Score-based attacks . . . 10
4.1.1 Zeroth-order gradient estimation . . . 10
4.1.2 Gradient-free random search . . . 11
4.2 Decision-based attacks . . . 11

5 Transfer-Based Black-Box Attacks **11**

6 Ensembles: Surrogates and Targets **12**

7 Evasion Attacks on Large Language Models **12**
7.1 Setting and threat model . . . 13
7.2 A surrogate loss and the discrete budget problem . . . 13
7.3 White-box: Greedy Coordinate Gradient . . . 14
7.4 Black-box attacks . . . 15
7.5 Prompt injection . . . 15
7.6 Defenses and their limits . . . 16

8 Hands-On: Implementing Evasion Attacks **16**
8.1 FGSM and PGD . . . 16
8.2 Carlini–Wagner ℓ_2 (single c, no binary search) . . . 17
8.3 A score-based attack: NES gradient estimate inside PGD . . . 17
8.4 Transfer from an ensemble of surrogates . . . 18
8.5 Suggested experiments . . . 18

Exercises **18**

Setting and notation

We keep the notation of Chapter 1: classifier f with logits $Z(\mathbf{x}) \in \mathbb{R}^K$, probabilities $F(\mathbf{x})$, loss $\mathcal{L}(\mathbf{x}, y)$ (we drop $\boldsymbol{\theta}$ since the model is fixed at inference time), threat set $\mathcal{B}_p(\mathbf{x}, \epsilon)$ and valid input domain $\mathcal{X} = [0,1]^d$. All attacks in this chapter solve, exactly or approximately, one of two problems:

$$\text{(budget)} \quad \max_{\mathbf{x}' \in \mathcal{B}_p(\mathbf{x},\epsilon) \cap \mathcal{X}} \mathcal{L}(\mathbf{x}', y) \qquad \text{(min-norm)} \quad \min_{\boldsymbol{\delta}} \|\boldsymbol{\delta}\|_p \ \text{ s.t. } \ f(\mathbf{x}+\boldsymbol{\delta}) \neq y,\ \mathbf{x}+\boldsymbol{\delta} \in \mathcal{X}.$$

Since both are optimization problems, *every* attack in this chapter is optimization-based, and "optimization-based" cannot name a family. What distinguishes attacks is the information the attacker has about f (gradients, scores, labels only, or nothing about the target at all), which problem is solved, and which solver is used. Section 2 organizes the chapter along these three axes. We denote by $\Pi_S(\cdot)$ the Euclidean projection onto a set S, and by $\text{clip}_{[0,1]}$ the elementwise projection onto the box.

1 Definitions: Evasion Attacks

1.1 Informal Definition

Evasion attacks craft inputs at inference time that cause the model to misbehave, without touching weights or training data.

1.2 Formal Definition

Let $\mathcal{Y} = \{1, \dots, K\}$ and let $f : \mathcal{X} \to \mathcal{Y}$ be the classifier, $f(\mathbf{x}) = \arg\max_k Z_k(\mathbf{x})$, with parameters $\boldsymbol{\theta}$ fixed. Let $\mathcal{D}$ be the data distribution over $\mathcal{X} \times \mathcal{Y}$. For a pair $(\mathbf{x}, y)$, write

$$\mathcal{S}_p(\mathbf{x}, \epsilon) \ = \ \mathcal{B}_p(\mathbf{x}, \epsilon) \cap \mathcal{X}$$

for the set of admissible inputs under budget ϵ.

Definition 1.1 (Adversarial example)**.** Let $\mathbf{x} \in \mathcal{X}$ with $f(\mathbf{x}) = y$. A point $\mathbf{x}'$ is an *adversarial example* for $(\mathbf{x}, y)$ at budget ϵ if $\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)$ and

$$\text{(untargeted)} \ \ f(\mathbf{x}') \neq y \qquad \text{or} \qquad \text{(targeted, } t \neq y) \ \ f(\mathbf{x}') = t.$$

Since f is an argmax, the untargeted condition is equivalent to $\max_{j \neq y} Z_j(\mathbf{x}') - Z_y(\mathbf{x}') \geq 0$ (up to tie-breaking), and the targeted one to $Z_t(\mathbf{x}') > \max_{j \neq t} Z_j(\mathbf{x}')$.

Definition 1.2 (Evasion attack)**.** An *evasion attack* is a (possibly randomized) algorithm $\mathcal{A}$ that receives $(\mathbf{x}, y)$ (and t if targeted), a threat set $\mathcal{B}_p(\mathbf{x}, \epsilon)$, and access to f only through an information level $\mathcal{I} \in \{\text{gradients}, \text{scores}, \text{labels}, \text{none}\}$ (white-box, score-based, decision-based, transfer), and returns

$$\mathbf{x}' = \mathcal{A}^{\mathcal{I}(f)}(\mathbf{x}, y; \epsilon) \in \mathcal{S}_p(\mathbf{x}, \epsilon).$$

The attack acts only on the input: $\boldsymbol{\theta}$ and the training set are fixed, and $\mathcal{A}$ is run at inference time. Level $\mathcal{I} = \text{none}$ means that no query to the target is made for the input being attacked; a surrogate model may still have been built beforehand, possibly by querying the target (Section 5).

Definition 1.3 (Attack success and robustness)**.** The *attack success rate* of $\mathcal{A}$ is

$$\text{ASR}(\mathcal{A}, \epsilon) = \Pr_{(\mathbf{x},y)\sim\mathcal{D}} \Big[f\big(\mathcal{A}^{\mathcal{I}(f)}(\mathbf{x}, y; \epsilon)\big) \neq y \ \Big|\ f(\mathbf{x}) = y \Big],$$

and the *robust accuracy* of f is

$$\text{RA}(f, \epsilon) = \Pr_{(\mathbf{x},y)\sim\mathcal{D}} \Big[\forall\, \mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon) :\ f(\mathbf{x}') = y \Big].$$

Remark 1.1 (Relation to the two optimization problems)**.** The *minimal adversarial perturbation*

$$\epsilon^\star(\mathbf{x}, y) = \min\{\|\boldsymbol{\delta}\|_p :\ f(\mathbf{x}+\boldsymbol{\delta}) \neq y,\ \mathbf{x}+\boldsymbol{\delta} \in \mathcal{X}\}$$

is the value of the (min-norm) problem. An adversarial example at budget ϵ exists if and only if $\epsilon^\star(\mathbf{x}, y) \leq \epsilon$. The (budget) problem replaces the indicator $\mathbb{1}[f(\mathbf{x}') \neq y]$, which has zero gradient almost everywhere, with the surrogate $\mathcal{L}(\mathbf{x}', y)$ (or a logit margin). Hence an attack that maximizes the surrogate need not find an adversarial example even when one exists, which is why the attacks below differ in surrogate, optimizer, and information level.

Remark 1.2 (Scope of the definition)**.** Definition 1.1 equates "admissible" with membership in an ℓ_p ball intersected with $\mathcal{X}$. This is a convenient, analyzable proxy for "imperceptible", not a model of what a real adversary can do. It does not cover semantic perturbations (rotations, translations, color shifts, style changes), physical-world perturbations (adversarial patches, stickers, printed or projected patterns), or domains where the input is not a continuous vector (text, graphs, malware binaries) and the threat set is defined by edit operations or functionality-preserving transformations. A small ℓ_p distance is neither necessary nor sufficient for an input to be "the same" to a human. All results in this chapter are therefore statements about the chosen threat set $\mathcal{B}_p(\mathbf{x}, \epsilon) \cap \mathcal{X}$, and robustness under one norm or budget does not imply robustness under another.

Remark 1.3 (Computability and bounds)**.** The robust accuracy $\mathrm{RA}(f, \epsilon)$ of Definition 1.3 contains a universal quantifier over a continuous set, so it is not computable by enumeration. Deciding whether an adversarial example exists is a nonconvex optimization problem, and for ReLU networks it is NP-hard in general (Katz et al., 2017). Exact verifiers exist (SMT or mixed-integer programming based), but they scale only to small networks. In practice one brackets RA from both sides.

Upper bound (attacks). Any attack $\mathcal{A}$ that returns $\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)$ exhibits a witness whenever $f(\mathbf{x}') \neq y$. Hence

$$\mathrm{RA}(f, \epsilon) \;\leq\; \mathrm{RA}_{\mathcal{A}}(f, \epsilon) \;=\; \Pr_{(\mathbf{x},y)\sim\mathcal{D}}\left[f(\mathcal{A}^{\mathcal{I}(f)}(\mathbf{x}, y; \epsilon)) = y\right].$$

A stronger attack lowers this bound, a weaker one (FGSM, an attack with gradient masking, too few iterations) leaves it loose. A *failed* attack is never a proof of robustness. Equivalently, every successful attack gives $\epsilon^\star(\mathbf{x}, y) \leq \|\mathbf{x}' - \mathbf{x}\|_p$, an upper bound on the minimal perturbation.

Lower bound (certification). A certification method returns, for each input, a radius $r(\mathbf{x}, y)$ such that f is provably constant on $\mathcal{B}_p(\mathbf{x}, r) \cap \mathcal{X}$, that is, $r \leq \epsilon^\star(\mathbf{x}, y)$. Counting the points with $r(\mathbf{x}, y) \geq \epsilon$ and $f(\mathbf{x}) = y$ gives

$$\mathrm{RA}_{\mathrm{cert}}(f, \epsilon) \;\leq\; \mathrm{RA}(f, \epsilon).$$

Typical routes are relaxation-based verification and verified training (Wong and Kolter, 2018; Gowal et al., 2019), which is deterministic but loose for large networks, and randomized smoothing (Cohen et al., 2019), which certifies a *smoothed* classifier g, not f itself, with a probabilistic guarantee for ℓ_2.

Consequence. The true value lies in the interval

$$\mathrm{RA}_{\mathrm{cert}}(f, \epsilon) \;\leq\; \mathrm{RA}(f, \epsilon) \;\leq\; \mathrm{RA}_{\mathcal{A}}(f, \epsilon),$$

and the width of the interval measures how much we do not know. A narrow gap is only achieved for small models or models trained for verifiability. For standard deep networks the certified lower bound is typically near zero while the attack upper bound is not, so reported numbers must state which side they bound. Two further caveats: all three quantities are estimated on a finite test set, so they carry sampling error in addition to the gap above, and the attack bound holds only if the returned point is actually checked to satisfy $\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)$ (see the implementation pitfalls).

2 A Taxonomy of Evasion Attacks

An attack is characterized by three largely independent choices.

Information level $\mathcal{I}$. What the attacker may observe about f (Definition 1.2). This is the axis that decides what is computable, and it is the top-level structure of this chapter: white-box attacks (Section 3), query-based black-box attacks with scores or labels (Section 4), and transfer-based black-box attacks (Section 5).

Problem solved. The (budget) problem fixes ϵ and maximizes a surrogate loss; it answers "can the model be fooled within this budget". The (min-norm) problem minimizes the perturbation size; it answers "how far is the nearest adversarial example", and returns a distance rather than a success rate at fixed ϵ.

Solver. How the optimization is carried out: projected first-order steps, penalty methods with an adaptive optimizer, iterative linearization, zeroth-order gradient estimation, random search, or boundary random walks.

Table 1 places the attacks of this chapter on these axes. The partition is by information level only; the other two columns cut across it. For example, C&W and PGD are both white-box gradient methods and differ in the problem solved, whereas PGD and NES-PGD solve the same problem and differ in the information level, and therefore in how the gradient is obtained.

Attack	Information	Problem	Solver	Norms
FGSM	gradients	budget	one linearized step	ℓ_∞, ℓ_2
BIM, PGD	gradients	budget	projected first-order ascent	ℓ_∞, ℓ_2
APGD	gradients	budget	PGD with adaptive step and momentum	ℓ_∞, ℓ_2
C&W	gradients	min-norm	penalty with margin loss, Adam, binary search on c	ℓ_0, ℓ_2, ℓ_∞
DeepFool	gradients	min-norm	iterative linearization of the boundaries	ℓ_2
ZOO	scores	min-norm (C&W-style)	coordinate-wise finite differences	ℓ_2
NES, SPSA	scores	budget	zeroth-order gradient estimate inside PGD	ℓ_∞
Square Attack	scores	budget	random search on the ball vertices	ℓ_∞
Boundary Attack	labels	min-norm	random walk along the boundary	ℓ_2
HopSkipJump	labels	min-norm	estimated boundary normal and binary search	ℓ_2, ℓ_∞
Transfer	none (per input)	either, on the surrogate	any white-box solver on a surrogate	any

Table 1: Attacks of this chapter along the three axes. Entries are a summary of the cited papers and some attacks have variants outside the listed norms.

Two caveats on the classification. First, transfer is a strategy rather than an algorithm: it chooses where the optimization is run (on a surrogate), not how. Second, some attacks straddle categories: the margin loss of C&W is reused inside PGD (budget problem), and NES also covers the partial-information and label-only settings (Ilyas et al., 2018).

3 White-Box Attacks

With $\mathcal{I}$ = gradients the attacker knows the architecture and the weights $\boldsymbol{\theta}$, and can compute derivatives of the model outputs with respect to the *input*. Concretely, for any $\mathbf{x}$ the attacker can evaluate the logits $Z(\mathbf{x}) \in \mathbb{R}^K$ and, by backpropagation, the gradients

$$\nabla_{\mathbf{x}} Z_k(\mathbf{x}) \in \mathbb{R}^d \quad (k = 1, \dots, K), \qquad \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, y) \in \mathbb{R}^d,$$

at the cost of one forward and one backward pass per gradient. The gradient is taken with respect to $\mathbf{x}$ and not with respect to $\boldsymbol{\theta}$: the parameters are fixed at inference time (Definition 1.2), so $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ plays no role here, in contrast with training. For a ReLU network the logits are piecewise linear in $\mathbf{x}$, so these gradients exist almost everywhere, but they may be zero or uninformative (saturated softmax, gradient masking), a point we return to when evaluating defenses.

The two problems of Section 1 use these quantities differently. Attacks on the budget problem (Definition 3.1) ascend the loss using $\nabla_{\mathbf{x}} \mathcal{L}$. Attacks on the min-norm problem (Definition 3.2) use the gradients of the logits, or of margins $m_k = Z_k - Z_y$, to locate and cross the decision boundary. We first treat the budget problem and then the min-norm problem.

3.1 Budget problem: FGSM, BIM and PGD

Definition 3.1 (Budget problem)**.** Let $(\mathbf{x}, y)$ with $f(\mathbf{x}) = y$, a norm $p \geq 1$ and a budget $\epsilon > 0$. The *budget problem* is

$$\max_{\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)} \mathcal{L}(\mathbf{x}', y) \;=\; \max_{\substack{\|\boldsymbol{\delta}\|_p \leq \epsilon \\ \mathbf{x}+\boldsymbol{\delta} \in \mathcal{X}}} \mathcal{L}(\mathbf{x} + \boldsymbol{\delta}, y), \tag{1}$$

and its targeted version, for a target class $t \neq y$, is $\min_{\mathbf{x}' \in \mathcal{S}_p(\mathbf{x}, \epsilon)} \mathcal{L}(\mathbf{x}', t)$, which is the same problem with $\mathcal{L}(\cdot, y)$ replaced by $-\mathcal{L}(\cdot, t)$.

The feasible set $\mathcal{S}_p(\mathbf{x}, \epsilon) = \mathcal{B}_p(\mathbf{x}, \epsilon) \cap \mathcal{X}$ is convex and compact, and the Euclidean projection onto it is cheap to compute or approximate. The objective is nonconcave in $\mathbf{x}'$, so problem (1) has no closed-form solution and may have many local maxima. The loss $\mathcal{L}$ is a differentiable surrogate for the indicator $\mathbb{1}[f(\mathbf{x}') \neq y]$ (Remark on the two optimization problems), so a high value of $\mathcal{L}$ does not by itself guarantee misclassification, and every candidate must be checked with f. The attacks of this subsection are three consecutive refinements of one idea, and we derive each from the previous one:

- **FGSM (Fast Gradient Sign Method)** replaces $\mathcal{L}$ by its first-order linearization at $\mathbf{x}$ and solves the resulting linear problem exactly, in one step.

- **BIM (Basic Iterative Method)** repeats this step with a small step size $\alpha < \epsilon$, re-linearizing at every iterate because the linear model is only trustworthy locally, and maps each iterate back to $\mathcal{S}_p(\mathbf{x}, \epsilon)$ by projection.
- **PGD (Projected Gradient Descent)** formalizes the repetition as projected steepest ascent for any ℓ_p norm, and adds random starts and restarts to cope with the nonconcavity of (1).

3.1.1 Fast Gradient Sign Method (FGSM)

Step 1: a local linear model of the loss. Let $\mathbf{g} = \nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}, y)$. A first-order Taylor expansion around $\mathbf{x}$ gives

$$\mathcal{L}(\mathbf{x} + \boldsymbol{\delta}, y) = \mathcal{L}(\mathbf{x}, y) + \boldsymbol{\delta}^\top \mathbf{g} + R(\boldsymbol{\delta}), \tag{2}$$

where the remainder $R(\boldsymbol{\delta})$ is of order $\|\boldsymbol{\delta}\|^2$ when $\mathcal{L}$ is twice differentiable. For a ReLU network the logits are piecewise linear in $\mathbf{x}$, so the linear model is exact inside the activation region that contains $\mathbf{x}$; the ball $\mathcal{B}_p(\mathbf{x}, \epsilon)$ may, however, cross many regions. Dropping R and the constant $\mathcal{L}(\mathbf{x}, y)$, the budget problem becomes

$$\max_{\|\boldsymbol{\delta}\|_p \le \epsilon} \boldsymbol{\delta}^\top \mathbf{g}. \tag{3}$$

This is the problem of finding the perturbation of size at most ϵ that is best aligned with $\mathbf{g}$, which depends on how size is measured.

Step 2: solving the linear problem.

Proposition 3.1 (Steepest ascent direction)**.** *For any $\mathbf{g} \in \mathbb{R}^d$, $\mathbf{g} \neq \mathbf{0}$, and $p \geq 1$, with $1/p + 1/q = 1$,*

$$\max_{\|\boldsymbol{\delta}\|_p \le \epsilon} \boldsymbol{\delta}^\top \mathbf{g} = \epsilon \|\mathbf{g}\|_q.$$

For $p = \infty$ ($q = 1$) the maximum is attained at $\boldsymbol{\delta} = \epsilon\ \mathrm{sign}(\mathbf{g})$; for $p = 2$ ($q = 2$) at $\boldsymbol{\delta} = \epsilon\, \mathbf{g}/\|\mathbf{g}\|_2$; for $1 < p < \infty$ at $\delta_i = \epsilon\ \mathrm{sign}(g_i)|g_i|^{q-1}/\|\mathbf{g}\|_q^{q-1}$.

Proof. Upper bound. Hölder's inequality states $\boldsymbol{\delta}^\top \mathbf{g} \leq \|\boldsymbol{\delta}\|_p \|\mathbf{g}\|_q$ for conjugate exponents $p, q \in [1, \infty]$. Hence $\boldsymbol{\delta}^\top \mathbf{g} \leq \epsilon \|\mathbf{g}\|_q$ for every feasible $\boldsymbol{\delta}$.

Attainment for $p = \infty$. Take $\boldsymbol{\delta} = \epsilon\ \mathrm{sign}(\mathbf{g})$. Then $\|\boldsymbol{\delta}\|_\infty = \epsilon$ and $\boldsymbol{\delta}^\top \mathbf{g} = \epsilon \sum_i |g_i| = \epsilon \|\mathbf{g}\|_1$, so the bound is tight.

Attainment for $1 < p < \infty$. Equality in Hölder's inequality requires $|\delta_i|^p$ proportional to $|g_i|^q$ with matching signs, which suggests $\delta_i = c\ \mathrm{sign}(g_i)|g_i|^{q-1}$. From $1/p + 1/q = 1$ we get $(q-1)p = q$. Therefore $\|\boldsymbol{\delta}\|_p^p = c^p \sum_i |g_i|^{(q-1)p} = c^p \|\mathbf{g}\|_q^q$, and $\|\boldsymbol{\delta}\|_p = \epsilon$ fixes $c = \epsilon / \|\mathbf{g}\|_q^{q/p} = \epsilon / \|\mathbf{g}\|_q^{q-1}$. The value is then $\boldsymbol{\delta}^\top \mathbf{g} = c \sum_i |g_i|^q = \epsilon \|\mathbf{g}\|_q^q / \|\mathbf{g}\|_q^{q-1} = \epsilon \|\mathbf{g}\|_q$. For $p = 2$ we have $q = 2$ and $c = \epsilon / \|\mathbf{g}\|_2$, which gives $\boldsymbol{\delta} = \epsilon\, \mathbf{g}/\|\mathbf{g}\|_2$. □

The case $p = 1$ ($q = \infty$) is Exercise 1. Two consequences are worth stating explicitly.

- For $p = \infty$ the optimal perturbation uses only the *sign* of each partial derivative, and the first-order gain is $\epsilon \|\mathbf{g}\|_1$. This sum has d terms, so for a fixed per-pixel budget ϵ the first-order gain grows with the input dimension. This is the linearity argument of Goodfellow et al. (2015): many individually negligible changes add up. It explains why the effect exists at first order, not that the linear model remains accurate.
- For the targeted attack we want to *decrease* the loss of the target class. Maximizing $-\mathcal{L}(\mathbf{x}, t)$ is the same problem with $\mathbf{g}$ replaced by $-\nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}, t)$, so the proposition gives $\boldsymbol{\delta} = -\epsilon\ \mathrm{sign}(\nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}, t))$.

Step 3: the box constraint. Problem (3) ignored $\mathbf{x} + \boldsymbol{\delta} \in [0, 1]^d$. For $p = \infty$ the constraints are separable: coordinate i must satisfy $\delta_i \in [l_i, u_i]$ with $l_i = -\min(\epsilon, x_i) \leq 0 \leq u_i = \min(\epsilon, 1 - x_i)$. The objective $\sum_i g_i \delta_i$ is linear in each coordinate, so the maximum sits at u_i if $g_i > 0$ and at l_i if $g_i < 0$. Since $x_i + u_i = \min(x_i + \epsilon, 1)$ and $x_i + l_i = \max(x_i - \epsilon, 0)$, this is exactly the sign step followed by clipping, so clipping loses nothing for ℓ_∞. The result is the attack of Goodfellow et al. (2015),

$$\mathbf{x}' = \mathrm{clip}_{[0,1]}\big(\mathbf{x} + \epsilon\ \mathrm{sign}(\nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}, y))\big). \tag{4}$$

For $p = 2$ the same reasoning with Proposition 3.1 gives the *fast gradient method* (FGM), $\boldsymbol{\delta} = \epsilon\, \mathbf{g}/\|\mathbf{g}\|_2$; here clipping is only an approximation of the box-constrained optimum, because the constraints no longer separate.

When can one linear step be trusted? Suppose $\nabla_{\mathbf{x}}\mathcal{L}$ is β-Lipschitz in the Euclidean norm. Then, writing $\mathbf{x}_t = \mathbf{x} + t\boldsymbol{\delta}$,

$$\mathcal{L}(\mathbf{x}+\boldsymbol{\delta}) - \mathcal{L}(\mathbf{x}) - \boldsymbol{\delta}^\top \mathbf{g} = \int_0^1 \big(\nabla\mathcal{L}(\mathbf{x}_t) - \nabla\mathcal{L}(\mathbf{x})\big)^\top \boldsymbol{\delta}\, dt \;\geq\; -\int_0^1 \beta t \|\boldsymbol{\delta}\|_2^2\, dt = -\tfrac{\beta}{2}\|\boldsymbol{\delta}\|_2^2.$$

For the FGSM perturbation $\|\boldsymbol{\delta}\|_2^2 = \epsilon^2 d$, so the actual loss increase is at least $\epsilon\|\mathbf{g}\|_1 - \frac{\beta}{2}\epsilon^2 d$. This guarantee is positive only if $\epsilon < 2\|\mathbf{g}\|_1/(\beta d)$: the single-step guarantee degrades with the curvature β and with the dimension. It is a lower bound, not a prediction, and deep networks are not globally smooth, so it should be read as intuition. It explains two observations at once. A model whose loss surface is highly curved near the data (large β, as in the gradient masking induced by FGSM training) makes the single step uninformative, and a smaller step that is re-linearized, as in BIM below, gives guarantees at a much smaller ϵ.

Analysis. FGSM costs one forward and one backward pass. It is exactly optimal only if the loss is linear in the ball, which it is not for deep networks. Its value today is as a fast baseline and as an inexpensive generator for adversarial training. It is a *weak* attack for evaluation: a model can resist FGSM while being completely broken by iterative attacks. In fact, models trained only on FGSM examples are known to exhibit *label leaking* and a form of gradient masking where the loss surface becomes highly curved near the data, making the single linear step uninformative (Tramèr et al., 2018).

3.1.2 Basic Iterative Method (BIM)

The remedy suggested by the previous analysis is to take a step small enough for the linear model to hold, move, and linearize again. Let $\mathbf{x}^{(k)}$ be the current iterate and $\mathbf{g}^{(k)} = \nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}^{(k)}, y)$. Within a trust region $\|\boldsymbol{\delta}\|_\infty \leq \alpha$ around $\mathbf{x}^{(k)}$, Proposition 3.1 says that the best step for the local linear model is $\alpha\ \mathrm{sign}(\mathbf{g}^{(k)})$, that is, an FGSM step with budget $\alpha < \epsilon$. After the step the iterate may have left the original ball, which is not allowed, so we map it back with the Euclidean projection Π onto $\mathcal{B}_\infty(\mathbf{x}, \epsilon) \cap [0,1]^d$, derived explicitly in the next subsection. This is the update of Kurakin et al. (2017):

$$\mathbf{x}^{(0)} = \mathbf{x}, \qquad \mathbf{x}^{(k+1)} = \Pi_{\mathcal{B}_\infty(\mathbf{x},\epsilon)\cap[0,1]^d}\Big(\mathbf{x}^{(k)} + \alpha\ \mathrm{sign}\left(\nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}^{(k)}, y)\right)\Big). \tag{5}$$

How many steps? Every step changes each coordinate by at most α, so after k steps $\|\mathbf{x}^{(k)} - \mathbf{x}\|_\infty \leq k\alpha$. Reaching the boundary of the ball therefore requires $T \geq \epsilon/\alpha$ iterations. With ϵ in 0–255 units and $\alpha = 1$ this reads $T \geq \epsilon$. Kurakin et al. use the heuristic $T = \lceil \min(\epsilon + 4,\ 1.25\epsilon) \rceil$, which adds slack to this lower bound. Our reading of the slack is that coordinates whose gradient sign flips between iterations move back, so more than ϵ/α steps are needed in practice, while the cap $\epsilon + 4$ keeps the cost bounded for large ϵ. For $\epsilon = 8$ the minimum is 8 steps and the heuristic gives $\lceil \min(12, 10) \rceil = 10$.

3.1.3 Projected Gradient Descent (PGD)

Projected steepest ascent. Let $S = \mathcal{B}_p(\mathbf{x}, \epsilon) \cap [0,1]^d$ be the feasible set. The generic method for maximizing a differentiable function over a closed set is: move along an ascent direction, then return to the set with the Euclidean projection $\Pi_S(\mathbf{z}) = \arg\min_{\mathbf{u}\in S} \|\mathbf{u} - \mathbf{z}\|_2$. The direction is a design choice. Plain gradient ascent uses $\mathbf{g}$; steepest ascent with respect to the ℓ_p norm uses the maximizer of $\mathbf{d}^\top\mathbf{g}$ over $\|\mathbf{d}\|_p \leq 1$, which Proposition 3.1 gives in closed form. For the two norms of this chapter,

$$\mathbf{x}^{(k+1)} = \Pi_S\big(\mathbf{x}^{(k)} + \alpha\, \mathbf{d}^{(k)}\big), \qquad \mathbf{d}^{(k)} = \begin{cases} \mathrm{sign}(\mathbf{g}^{(k)}) & p = \infty, \\ \mathbf{g}^{(k)}/\|\mathbf{g}^{(k)}\|_2 & p = 2. \end{cases}$$

BIM is this iteration for $p = \infty$. What Madry et al. (2018) add is the *starting point*, discussed below.

The projections. Both are short derivations.

Proposition 3.2 (Projection for ℓ_∞)**.** *Let* $S = \mathcal{B}_\infty(\mathbf{x}, \epsilon) \cap [0,1]^d$. *Then* $\Pi_S(\mathbf{z})_i = \min\big(\max(z_i,\ x_i - \epsilon,\ 0),\ x_i + \epsilon,\ 1\big)$.

Proof. The set S is the box $\{\mathbf{u} :\ a_i \leq u_i \leq b_i\}$ with $a_i = \max(x_i - \epsilon, 0)$ and $b_i = \min(x_i + \epsilon, 1)$, because the intersection of two boxes is a box. The objective $\|\mathbf{u} - \mathbf{z}\|_2^2 = \sum_i (u_i - z_i)^2$ is a sum of terms that each involve one coordinate, and the constraints also involve one coordinate each, so the problem splits into d scalar problems $\min_{a_i \leq u_i \leq b_i} (u_i - z_i)^2$. The minimizer of a convex parabola centered at z_i over an interval is z_i if it lies in the interval and the nearest endpoint otherwise, which is $\min(\max(z_i, a_i), b_i)$. □

Proposition 3.3 (Projection onto an ℓ_2 ball)**.** *For* $S = \mathcal{B}_2(\mathbf{x}, \epsilon)$, $\Pi_S(\mathbf{z}) = \mathbf{x} + \epsilon\,\mathbf{v}/\max(\epsilon, \|\mathbf{v}\|_2)$ *with* $\mathbf{v} = \mathbf{z} - \mathbf{x}$.

Proof. Write $\mathbf{u} = \mathbf{x} + \mathbf{w}$ with $\|\mathbf{w}\|_2 \le \epsilon$, so the problem is $\min \|\mathbf{w} - \mathbf{v}\|_2$. If $\|\mathbf{v}\|_2 \le \epsilon$, then $\mathbf{w} = \mathbf{v}$ is feasible with value zero. Otherwise, the triangle inequality gives $\|\mathbf{w} - \mathbf{v}\|_2 \ge \|\mathbf{v}\|_2 - \|\mathbf{w}\|_2 \ge \|\mathbf{v}\|_2 - \epsilon$, with equality exactly when $\mathbf{w}$ points in the direction of $\mathbf{v}$ and $\|\mathbf{w}\|_2 = \epsilon$, that is, $\mathbf{w} = \epsilon\mathbf{v}/\|\mathbf{v}\|_2$. □

For ℓ_2 the feasible set also includes the box, and the intersection of a ball and a box is not a box, so the problem no longer splits. Rescaling and then clipping is therefore only an approximation of Π_S, which is the usual practice (Exercise 2).

Why random starts and restarts. The budget problem is nonconcave, so the iteration converges (if at all) to a local maximum determined by its starting point. Starting from $\mathbf{x}$ itself is a deterministic choice: if the gradient there is uninformative, for example because cross-entropy saturates on a very confident input, every restart from $\mathbf{x}$ fails in the same way. Sampling the start $\mathbf{x}^{(0)} = \mathrm{clip}_{[0,1]}(\mathbf{x} + \mathbf{u})$, $\mathbf{u} \sim \mathcal{U}[-\epsilon, \epsilon]^d$, places the iterate in different basins of attraction. Running R restarts and keeping the best of them gives a maximum over R local maxima, which cannot decrease as R grows. This is the reason for the sanity check below that more restarts should never raise the reported accuracy.

Algorithm 1 PGD attack, ℓ_∞, untargeted

Require: $\mathbf{x}, y$, budget ϵ, step size α, iterations T, restarts R

$\mathbf{x}'_{\text{best}} \leftarrow \mathbf{x}$
for $r = 1, \dots, R$ **do**
  $\mathbf{x}^{(0)} \leftarrow \mathrm{clip}_{[0,1]}(\mathbf{x} + \mathbf{u})$, $\mathbf{u} \sim \mathcal{U}[-\epsilon, \epsilon]^d$
  for $k = 0, \dots, T-1$ **do**
    $\mathbf{g} \leftarrow \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^{(k)}, y)$
    $\mathbf{x}^{(k+1)} \leftarrow \Pi_{\mathcal{B}_\infty(\mathbf{x},\epsilon) \cap [0,1]^d}\big(\mathbf{x}^{(k)} + \alpha\ \mathrm{sign}(\mathbf{g})\big)$
  end for
  keep $\mathbf{x}^{(T)}$ as $\mathbf{x}'_{\text{best}}$ if it misclassifies (or has higher loss)
end for
return $\mathbf{x}'_{\text{best}}$

Why PGD became the reference first-order attack. Madry et al. (2018) observed empirically that, from many random starts, PGD finds local maxima of the loss with remarkably concentrated values. This suggests that, for first-order methods, PGD is close to the best one can do, which motivated using PGD both to evaluate models and to train them. This is an empirical observation, not a theorem: the inner problem is nonconcave and PGD offers no global guarantee.

Practical analysis.

- **Step size and iterations.** Too small an α with few iterations never reaches the boundary (the bound $T \ge \epsilon/\alpha$ above). Too large an α wastes the iteration: if $\alpha \ge 2\epsilon$, then after the random start each coordinate offset lies in $[-\epsilon, \epsilon]$, the sign step moves it by $\pm\alpha$, and the projection sends it to $\pm\epsilon$. Every iterate is then a vertex of the ball, the update can no longer refine the solution, and the method degenerates into hopping among vertices. A typical CIFAR-10 setting is $\epsilon = 8/255$, $\alpha = 2/255$, $T \in [20, 100]$.
- **Loss choice.** Cross-entropy saturates when the model is very confident, giving tiny gradients. Margin losses on logits, such as $\max_{j \ne y} Z_j(\mathbf{x}) - Z_y(\mathbf{x})$ (Section 3.2.1), or the difference-of-logits-ratio loss, are often stronger.
- **Fixed schedules are brittle.** APGD (Croce and Hein, 2020) adapts the step size and adds momentum, removing the hyperparameter sensitivity of PGD. Together with complementary attacks, it forms AutoAttack, the current standard for reporting ℓ_∞ and ℓ_2 robustness (Chapter 6).

Reading PGD results

A PGD robust accuracy is an *upper bound* on true robust accuracy (Remark 1.3). Sanity checks: accuracy should decrease monotonically with ϵ and reach zero for large ϵ; more iterations and restarts should not increase accuracy; PGD should beat FGSM; white-box should beat black-box. Violations indicate gradient masking or a bug (Carlini et al., 2019).

3.2 Min-norm problem: Carlini and Wagner, and DeepFool

Definition 3.2 (Min-norm problem)**.** Let $(\mathbf{x}, y)$ with $f(\mathbf{x}) = y$ and a norm $p \geq 1$. The *min-norm problem* is

$$\epsilon^\star(\mathbf{x}, y) \;=\; \min_{\boldsymbol{\delta}} \|\boldsymbol{\delta}\|_p \quad \text{s.t.} \quad f(\mathbf{x}+\boldsymbol{\delta}) \neq y, \;\; \mathbf{x}+\boldsymbol{\delta} \in \mathcal{X}, \tag{6}$$

and its targeted version, for a target class $t \neq y$, replaces the constraint $f(\mathbf{x}+\boldsymbol{\delta}) \neq y$ by $f(\mathbf{x}+\boldsymbol{\delta}) = t$. An attack returns a feasible $\boldsymbol{\delta}$, and its norm $\|\boldsymbol{\delta}\|_p$ is an upper bound on $\epsilon^\star(\mathbf{x}, y)$.

Problem (6) has no budget ϵ: the output is a *distance*, not a success rate at a fixed ϵ. It is the dual view of the budget problem (Definition 3.1): an adversarial example at budget ϵ exists if and only if $\epsilon^\star(\mathbf{x}, y) \leq \epsilon$ (Remark on the two optimization problems). The objective $\|\boldsymbol{\delta}\|_p$ is convex, but the feasible set $\{\boldsymbol{\delta} : f(\mathbf{x}+\boldsymbol{\delta}) \neq y\} \cap (\mathcal{X} - \mathbf{x})$ is in general nonconvex, since it is the complement of the (possibly nonconvex) region where the model predicts y. Moreover, the constraint is discrete, so it provides no gradient, and the box constraint $\mathbf{x}+\boldsymbol{\delta} \in [0,1]^d$ is awkward for gradient methods. The attacks of this subsection differ in how they deal with these obstacles:

- **C&W** (Carlini and Wagner, 2017) replaces the discrete constraint by a penalty on a logit margin, removes the box constraint by a change of variables, solves the resulting unconstrained problem with Adam, and selects the penalty weight per input by binary search. It exists for ℓ_0, ℓ_2 and ℓ_∞.
- **DeepFool** (Moosavi-Dezfooli et al., 2016) is a cheaper alternative for ℓ_2: at each step it linearizes all class boundaries around the current iterate and jumps to the nearest one. For a linear classifier this recovers the exact minimal ℓ_2 perturbation (compare Exercise 2 of Chapter 1).

We develop C&W in detail, since it introduces the margin surrogate that is reused elsewhere in the chapter.

3.2.1 Carlini and Wagner

Carlini and Wagner (2017) (C&W) attack problem (6) directly, and must remove the two obstacles named above: the constraint $f(\mathbf{x}+\boldsymbol{\delta}) = t$ is discrete, and the box constraint $\mathbf{x}+\boldsymbol{\delta} \in [0,1]^d$ is awkward for gradient methods.

From a hard constraint to a penalty. Replace the classification constraint by a function h such that $f(\mathbf{x}+\boldsymbol{\delta}) = t$ holds whenever $h(\mathbf{x}+\boldsymbol{\delta}) \leq 0$. After comparing several candidates, C&W recommend the logit margin

$$h(\mathbf{x}') = \max\Big(\max_{j \neq t} Z_j(\mathbf{x}') - Z_t(\mathbf{x}'),\; -\kappa\Big), \tag{7}$$

where $\kappa \geq 0$ is a *confidence* parameter: the attack keeps pushing until the target logit exceeds every other logit by at least κ. For the untargeted version, use $\max(Z_y - \max_{j \neq y} Z_j, -\kappa)$. Working with logits instead of softmax probabilities avoids saturation.

Removing the box constraint. Change variables to $\mathbf{w} \in \mathbb{R}^d$ with

$$\mathbf{x}' = \tfrac{1}{2}\big(\tanh(\mathbf{w}) + 1\big),$$

so that $\mathbf{x}' \in [0,1]^d$ automatically and the problem becomes unconstrained in $\mathbf{w}$.

The ℓ_2 attack. Combining both,

$$\min_{\mathbf{w}} \; \big\|\tfrac{1}{2}(\tanh(\mathbf{w}) + 1) - \mathbf{x}\big\|_2^2 + c \cdot h\big(\tfrac{1}{2}(\tanh(\mathbf{w}) + 1)\big), \tag{8}$$

solved with Adam. The constant $c > 0$ trades off distortion against success; it is chosen per input by *binary search*: increase c when the attack fails, decrease it when it succeeds, and keep the smallest successful perturbation.

ℓ_0 and ℓ_∞ variants. The ℓ_∞ norm is non-differentiable and gradient descent on it oscillates, so C&W replace it by the penalty $\sum_i \max(|\delta_i| - \tau, 0)$ and shrink τ over iterations. The ℓ_0 attack repeatedly runs the ℓ_2 attack and freezes the pixels that contributed least, shrinking the set of modifiable features.

Analysis.

- C&W produces much smaller perturbations than FGSM/PGD at a fixed success rate and was historically decisive in breaking defenses such as defensive distillation, which shrank softmax gradients but left logits informative.
- It is expensive: thousands of gradient steps per input including the binary search.
- To compare with PGD, count the fraction of inputs whose minimal C&W perturbation is below ϵ.
- The margin loss (Eq. 7) is widely reused inside PGD ("PGD-CW"), often outperforming cross-entropy.

3.2.2 DeepFool

C&W solves a penalized problem with thousands of gradient steps. DeepFool (Moosavi-Dezfooli et al., 2016) instead exploits the fact that the min-norm problem has a closed-form solution when the classifier is linear, and applies that solution repeatedly to the local linearization of a nonlinear one. We write the derivation for ℓ_2 and state the general ℓ_p case afterwards.

Step 1: the linear binary case. Let $g(\mathbf{x}) = \mathbf{a}^\top \mathbf{x} + b$ with $f(\mathbf{x}) = \operatorname{sign}(g(\mathbf{x}))$, $\mathbf{a} \neq \mathbf{0}$.

Proposition 3.4 (Distance to a hyperplane)**.** *The minimal ℓ_2 perturbation that changes the sign of g at $\mathbf{x}$ has norm $|g(\mathbf{x})|/\|\mathbf{a}\|_2$, and is attained at*

$$\boldsymbol{\delta}^\star = -\frac{g(\mathbf{x})}{\|\mathbf{a}\|_2^2}\,\mathbf{a}.$$

Proof. Changing the sign requires $g(\mathbf{x}+\boldsymbol{\delta}) = g(\mathbf{x}) + \mathbf{a}^\top\boldsymbol{\delta}$ to reach zero or beyond, so $|\mathbf{a}^\top\boldsymbol{\delta}| \geq |g(\mathbf{x})|$ with $\mathbf{a}^\top\boldsymbol{\delta}$ of sign opposite to $g(\mathbf{x})$. By Cauchy–Schwarz, $|\mathbf{a}^\top\boldsymbol{\delta}| \leq \|\mathbf{a}\|_2\|\boldsymbol{\delta}\|_2$, hence $\|\boldsymbol{\delta}\|_2 \geq |g(\mathbf{x})|/\|\mathbf{a}\|_2$. Equality in Cauchy–Schwarz holds when $\boldsymbol{\delta}$ is parallel to $\mathbf{a}$, which gives $\boldsymbol{\delta}^\star$, and then $g(\mathbf{x}+\boldsymbol{\delta}^\star) = 0$. □

The perturbation is the orthogonal projection of $\mathbf{x}$ onto the hyperplane $\{g = 0\}$. It is the same Hölder argument as in Proposition 3.1, read as a min-norm problem instead of a budget problem.

Step 2: the linear multiclass case. Let $Z_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + b_k$ and $f(\mathbf{x}) = y$. For each $k \neq y$ define the margin

$$m_k(\mathbf{x}) = Z_k(\mathbf{x}) - Z_y(\mathbf{x}) = \mathbf{a}_k^\top \mathbf{x} + \beta_k, \qquad \mathbf{a}_k = \mathbf{w}_k - \mathbf{w}_y, \quad \beta_k = b_k - b_y,$$

which is negative at $\mathbf{x}$ because $f(\mathbf{x}) = y$. The region where the model predicts y is the polyhedron $P = \{\mathbf{x} : m_k(\mathbf{x}) < 0\ \forall k \neq y\}$, and its complement is the union of the half-spaces $H_k = \{m_k \geq 0\}$. Hence

$$\epsilon^\star(\mathbf{x}, y) = \min_{k \neq y} \operatorname{dist}_2(\mathbf{x}, H_k) = \min_{k \neq y} \frac{|m_k(\mathbf{x})|}{\|\mathbf{a}_k\|_2}, \tag{9}$$

using Proposition 3.4 for each k. The minimizer is the class l whose boundary is closest, and the optimal perturbation is

$$\boldsymbol{\delta}^\star = \frac{|m_l(\mathbf{x})|}{\|\mathbf{a}_l\|_2^2}\,\mathbf{a}_l, \qquad l = \arg\min_{k \neq y} \frac{|m_k(\mathbf{x})|}{\|\mathbf{a}_k\|_2}.$$

This is exact when the box constraint is inactive. The nearest boundary in terms of *logit gap* $|m_k|$ is not in general the nearest in terms of *distance*, because the denominators $\|\mathbf{a}_k\|_2$ differ.

Step 3: nonlinear classifiers. For a differentiable Z, replace m_k around the current iterate $\mathbf{x}^{(i)}$ by its first-order model

$$m_k(\mathbf{x}^{(i)} + \boldsymbol{\delta}) \approx m_k(\mathbf{x}^{(i)}) + \nabla m_k(\mathbf{x}^{(i)})^\top \boldsymbol{\delta}, \qquad \nabla m_k = \nabla Z_k - \nabla Z_y,$$

which is the linear multiclass classifier of Step 2 with $\mathbf{a}_k = \nabla m_k(\mathbf{x}^{(i)})$. One DeepFool step applies the formula above to this model:

$$l_i = \arg\min_{k \neq y} \frac{|m_k(\mathbf{x}^{(i)})|}{\|\nabla m_k(\mathbf{x}^{(i)})\|_2}, \qquad \mathbf{r}_i = \frac{|m_{l_i}(\mathbf{x}^{(i)})|}{\|\nabla m_{l_i}(\mathbf{x}^{(i)})\|_2^2}\,\nabla m_{l_i}(\mathbf{x}^{(i)}). \tag{10}$$

Since the linear model is only local, the iteration is repeated from the new point until the label changes. The final perturbation is the accumulated sum of the steps.

The overshoot $\eta > 0$ (the original paper uses $\eta = 0.02$) is needed because each step lands *on* the linearized boundary, and the true boundary is only approximated. Without it the iterate often stops at $m_l \approx 0^-$ and the label does not change. The update recomputes the point from $\mathbf{x}$ and the accumulated perturbation, so the overshoot does not compound across iterations.

General ℓ_p. The same derivation with Hölder's inequality in place of Cauchy–Schwarz gives, for $1/p + 1/q = 1$, the distance $|m_k|/\|\nabla m_k\|_q$ and the step

$$\mathbf{r}_i = \frac{|m_l(\mathbf{x}^{(i)})|}{\|\nabla m_l(\mathbf{x}^{(i)})\|_q^q}\,\operatorname{sign}(\nabla m_l)\,|\nabla m_l|^{q-1}.$$

For $p = \infty$ ($q = 1$) this reduces to $\mathbf{r}_i = |m_l|\,\operatorname{sign}(\nabla m_l)/\|\nabla m_l\|_1$, which is again the sign step of Proposition 3.1, now with a step size chosen to land on the linearized boundary instead of a fixed budget.

Algorithm 2 DeepFool, ℓ_2, untargeted

Require: $\mathbf{x}, y = f(\mathbf{x})$, overshoot η, maximum iterations T

$\mathbf{x}^{(0)} \leftarrow \mathbf{x}$, $\mathbf{r}_{\text{tot}} \leftarrow \mathbf{0}$
for $i = 0, \dots, T-1$ **do**
  if $f(\mathbf{x}^{(i)}) \neq y$ **then break**
  end if
  compute $m_k(\mathbf{x}^{(i)})$ and $\nabla m_k(\mathbf{x}^{(i)})$ for all $k \neq y$ ▷ $K-1$ backward passes
  compute l_i and $\mathbf{r}_i$ by Eq. (10)
  $\mathbf{r}_{\text{tot}} \leftarrow \mathbf{r}_{\text{tot}} + \mathbf{r}_i$
  $\mathbf{x}^{(i+1)} \leftarrow \text{clip}_{[0,1]}\big(\mathbf{x} + (1+\eta)\,\mathbf{r}_{\text{tot}}\big)$
end for
return $\mathbf{x}^{(i)}$ and $\|\mathbf{x}^{(i)} - \mathbf{x}\|_2$

Analysis.

- DeepFool is exact for linear classifiers (Eq. 9) and is a fixed-point iteration of local linearizations for nonlinear ones. It has no optimality guarantee: the returned norm is an upper bound on $\epsilon^\star(\mathbf{x}, y)$, as for any attack.
- It needs no penalty weight and no binary search, and typically converges in a small number of iterations. The cost per iteration is up to $K-1$ backward passes, which can be reduced by restricting the search to the few classes with the largest logits.
- It tends to return larger perturbations than C&W on the same input, since C&W optimizes the distortion directly. Its role is a fast estimate of the distance to the boundary, for example to compute an average robustness over a dataset, and not the strongest attack for evaluating a defense.
- The box constraint is handled by clipping, which is only an approximation of the constrained problem, and the ℓ_2 projection issue of Exercise 2 applies here as well.
- Because it linearizes the logits, it is affected by gradient masking in the same way as FGSM: if ∇m_k is uninformative, the step is unreliable.

4 Query-Based Black-Box Attacks

Here the attacker cannot differentiate through f but can query it on chosen inputs. The cost of an attack is measured in queries. We distinguish what a query returns: scores (Section 4.1) or only the predicted label (Section 4.2).

4.1 Score-based attacks

Suppose the attacker can query $F(\mathbf{x})$ (or the top-k scores) but has no gradients. The loss $\mathcal{L}(\mathbf{x}, y)$ is still computable from scores, so the budget problem can still be attacked, provided we can *estimate* the gradient or avoid it.

4.1.1 Zeroth-order gradient estimation

Coordinate-wise finite differences. ZOO (Chen et al., 2017) estimates partial derivatives with symmetric differences,

$$\hat{g}_i = \frac{\mathcal{L}(\mathbf{x} + h\mathbf{e}_i, y) - \mathcal{L}(\mathbf{x} - h\mathbf{e}_i, y)}{2h},$$

updating a random batch of coordinates per iteration (with a C&W-style objective). An exact gradient needs $2d$ queries, which is prohibitive for images; ZOO mitigates this by attacking in a downsampled space and prioritizing important coordinates.

Random-direction estimators (NES). Ilyas et al. (2018) use natural evolution strategies. They sample n directions $\mathbf{u}_i \sim \mathcal{N}(0, I)$ and estimate

$$\hat{\mathbf{g}} = \frac{1}{2n\sigma} \sum_{i=1}^{n} \big[\mathcal{L}(\mathbf{x} + \sigma\mathbf{u}_i, y) - \mathcal{L}(\mathbf{x} - \sigma\mathbf{u}_i, y)\big]\mathbf{u}_i, \tag{11}$$

which is an unbiased estimate of the gradient of a Gaussian-smoothed loss. Plugging $\hat{\mathbf{g}}$ into PGD gives an effective attack with far fewer than d queries per step. The same paper handles the *partial-information* (top-k only) and *label-only* settings. SPSA (Uesato et al., 2018) is a closely related estimator using Rademacher directions; it is also a standard tool to test for gradient masking, since it does not rely on the model's backpropagated gradients.

4.1.2 Gradient-free random search

Square Attack. Andriushchenko et al. (2020) dispense with gradient estimation. At each iteration a random square window is chosen and the perturbation inside it is resampled at the vertices of the ℓ_∞ ball (values $\pm\epsilon$); the change is accepted only if the margin loss decreases. Window size shrinks over time. Despite its simplicity, Square Attack is among the most query-efficient score-based attacks and, because it never touches gradients, is immune to gradient masking. It is part of AutoAttack for that reason.

Analysis. Score-based attacks reveal that hiding gradients is not a defense: if scores are exposed, gradients can be approximated. Their cost is measured in queries, typically hundreds to a few thousand per image for untargeted ℓ_∞ attacks on CIFAR-scale models, more for targeted attacks and larger inputs. Defenses that only return labels, add noise to scores, or rate-limit queries raise the cost but do not eliminate the threat.

4.2 Decision-based attacks

In the hardest query setting the attacker sees only $f(\mathbf{x})$. The loss is now a step function, and its finite differences are zero almost everywhere. Decision-based attacks therefore work with the geometry of the decision boundary directly, and naturally solve the *min-norm* problem.

Boundary Attack. Brendel et al. (2018) start from a point that is already adversarial (for untargeted attacks, random noise that is misclassified; for targeted attacks, an image of the target class) and perform a random walk *along* the boundary that progressively reduces the distance to $\mathbf{x}$. Each step combines an orthogonal random perturbation (exploring along the boundary) and a small step toward $\mathbf{x}$, and is accepted only if the point remains adversarial. Step sizes are adapted from the acceptance rate. It needs only labels but can require tens of thousands of queries.

HopSkipJump. Chen et al. (2020) improve query efficiency by estimating the gradient direction *at the boundary* from binary feedback. At a boundary point $\mathbf{x}'_b$, define $\phi(\mathbf{z}) = +1$ if $\mathbf{z}$ is adversarial and -1 otherwise. Then

$$\widehat{\nabla} = \frac{1}{n}\sum_{i=1}^{n} \phi(\mathbf{x}'_b + \sigma \mathbf{u}_i)\, \mathbf{u}_i$$

points, in expectation and for small σ, along the boundary normal. Each iteration: (i) estimate the normal, (ii) take a geometric step along it, (iii) binary-search back to the boundary on the segment toward $\mathbf{x}$. It supports both ℓ_2 and ℓ_∞ and converges with far fewer queries than the Boundary Attack.

Information versus cost

White-box	exact gradients	tens of forward/backward passes
Score-based	loss values	10^2–10^4 queries
Decision-based	labels	10^3–10^5 queries
Transfer	nothing from the target per input	no per-input queries, lower success

Less information means higher cost and, usually, larger perturbations, but almost never complete protection.

5 Transfer-Based Black-Box Attacks

Szegedy et al. (2014) already noted that adversarial examples crafted for one network often fool another network trained on different data or with a different architecture. Papernot et al. (2017) turned this into a practical black-box attack: train a *substitute* model by querying the target on synthetic inputs (using Jacobian-based dataset augmentation), attack the substitute with white-box methods, and submit the result to the target. The queries are spent once to build the substitute, not per attacked input. If a surrogate is available without any access to the target (a public pretrained model, for example), no query is needed at all.

Why transfer works. Two complementary explanations:

- **Shared subspaces.** Tramèr et al. (2017) estimate that adversarial perturbations span a subspace of considerable dimension around each input, and that the decision boundaries of different models are close and aligned within it, so perturbations crossing one boundary tend to cross the other.
- **Shared non-robust features.** If models trained on the same distribution learn the same brittle but predictive features (Ilyas et al., 2019), a perturbation that manipulates those features affects all of them.

What makes examples transfer better. Strong white-box optimization tends to *overfit* to the surrogate: highly specific perturbations exploit idiosyncrasies of one model. Empirically:

- Untargeted examples transfer far better than targeted ones (Liu et al., 2017).
- Higher-confidence examples (larger κ in C&W) transfer better than minimal ones.
- *Momentum* stabilizes the update direction and reduces overfitting to the surrogate (MI-FGSM, Dong et al., 2018):

$$\mathbf{m}^{(k+1)} = \mu\,\mathbf{m}^{(k)} + \frac{\nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}^{(k)}, y)}{\|\nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}^{(k)}, y)\|_1}, \qquad \mathbf{x}^{(k+1)} = \Pi\big(\mathbf{x}^{(k)} + \alpha\ \mathrm{sign}(\mathbf{m}^{(k+1)})\big).$$

- *Input diversity*: applying random resizing and padding before each gradient computation acts like data augmentation for the attack (Xie et al., 2019).
- Attacking an *ensemble* of surrogates (next section).

Transfer and evaluation

Transfer success depends heavily on surrogate choice; low transfer success is not evidence of robustness. Conversely, if a transfer attack beats your white-box attack on a defended model, the white-box attack is failing, which is a classic sign of gradient masking.

6 Ensembles: Surrogates and Targets

Ensembles appear in this chapter in two roles: as surrogates that improve transfer, and as targets that claim robustness. In neither role do they define a new attack family; they change the function being attacked.

Ensembles as surrogates. Liu et al. (2017) showed that optimizing a perturbation against several models simultaneously yields examples that transfer much better, including targeted examples, to an unseen model. For models $f_1, \ldots, f_M$ with weights $w_m \geq 0$, $\sum_m w_m = 1$, the common fusion choices are

$$\begin{aligned} \text{logit fusion:} \quad & Z_{\text{ens}}(\mathbf{x}) = \textstyle\sum_m w_m Z_m(\mathbf{x}), \\ \text{probability fusion:} \quad & F_{\text{ens}}(\mathbf{x}) = \textstyle\sum_m w_m F_m(\mathbf{x}), \\ \text{loss fusion:} \quad & \mathcal{L}_{\text{ens}}(\mathbf{x}, y) = \textstyle\sum_m w_m \mathcal{L}_m(\mathbf{x}, y). \end{aligned}$$

Logit fusion is usually the most effective for transfer. The intuition: a direction that fools many diverse models is more likely to exploit features shared by all models, including the unseen target.

Ensembles as defenses. An ensemble is a single differentiable function. If the defender averages logits or probabilities, a white-box attacker simply attacks the averaged output with PGD, exactly as for a single model. Problems arise only with *non-differentiable* aggregation (majority vote) or with *randomized* selection of members. In those cases:

- Replace the vote with a differentiable surrogate (average of logits or probabilities) for gradient computation; the resulting example usually fools the vote as well.
- For random member selection, optimize the *expected* loss, $\mathbb{E}_m[\mathcal{L}_m(\mathbf{x}', y)]$, i.e., Expectation over Transformation (Athalye et al., 2018).
- For ensembles of detectors or heterogeneous defenses, attack all components jointly with a combined loss. He et al. (2017) showed that combining several weak defenses does not yield a strong one under adaptive attack.

Ensemble adversarial training (Tramèr et al., 2018) is a different idea: augmenting training with adversarial examples transferred from other, fixed models. It improves robustness to *transfer* attacks specifically, not to white-box attacks on the defended model.

7 Evasion Attacks on Large Language Models

The framework of Section 1 was built for classifiers with continuous inputs. Large language models (LLMs) break its assumptions in four ways, and each one forces a change in the definitions or in the algorithms.

1. **Discrete inputs.** The input is a sequence of tokens, so $\mathbf{x} + \boldsymbol{\delta}$ is not an input and gradient steps do not stay inside the domain.
2. **Structured outputs.** The model produces a sequence by sampling, not a label. "Misbehaving" is a predicate on the whole output, usually evaluated by a judge.

3. **No ℓ_p ball.** The admissible set is defined by what the attacker controls (a suffix, a document in a retrieval index, a conversation turn) or by preserving the meaning of a request, not by a distance.
4. **Data and instructions share one channel.** Everything is text in one context window, so an attacker who controls any part of the context can issue instructions (Vassilev et al., 2025). This is the structural analogue of SQL injection.

We keep the three-axis taxonomy of Section 2. What changes is the threat set, the surrogate loss, and the solver.

7.1 Setting and threat model

Let $\mathcal{V}$ be the vocabulary, $N = |\mathcal{V}|$, and let $\mathbf{x} \in \mathcal{V}^n$ be a token sequence. The model defines an autoregressive distribution over responses $\mathbf{r} = (r_1, \ldots, r_m)$,

$$p(\mathbf{r} \mid \mathbf{x}) = \prod_{t=1}^{m} p(r_t \mid \mathbf{x}, r_{<t}).$$

A *policy* is a predicate $J(\mathbf{x}, \mathbf{r}) \in \{0, 1\}$ that equals 1 when the response is unacceptable for that prompt (a harmful completion, a leaked system prompt, an action the user did not request). In practice J is a human, a classifier, or another LLM, and this has consequences for evaluation (see the warning box at the end of the section).

Definition 7.1 (Evasion against an LLM)**.** Let q be a request that the model should not fulfill (so $J(q, \mathbf{r}) = 1$ for compliant responses). Let $\mathcal{T}(q) \subseteq \mathcal{V}^*$ be a set of admissible transformations of q (the threat set). An attack returns $\mathbf{x}' \in \mathcal{T}(q)$, and it *succeeds* if

$$\Pr_{\mathbf{r} \sim p(\cdot \mid \mathbf{x}')} \big[J(q, \mathbf{r}) = 1 \big] \geq \tau$$

for a chosen threshold τ (for example, τ small for "at least one sample succeeds" and close to 1 for "reliably").

The threat set $\mathcal{T}$ replaces $\mathcal{B}_p(\mathbf{x}, \epsilon) \cap \mathcal{X}$. The common choices are:

- **Suffix (or prefix) of length k:** $\mathcal{T}(q) = \{q \oplus \mathbf{s} : \mathbf{s} \in \mathcal{V}^k\}$, where $\oplus$ is concatenation. The budget is the length k.
- **Intent-preserving rewrites:** $\mathcal{T}(q) = \{\mathbf{x} : \ \mathbf{x}$ asks for the same thing as $q\}$. This is the analogue of "imperceptible", and unlike $\|\boldsymbol{\delta}\|_p \leq \epsilon$ it cannot be checked by computing a norm. Whether a rewrite preserves intent is itself a judgment.
- **Attacker-controlled context:** the attacker does not write the user request at all. They control a document d that the application places into the context (indirect prompt injection, Section 7.5).

Two things from the classifier setting disappear. There is no clean pair $(\mathbf{x}, y)$ with $f(\mathbf{x}) = y$ to start from, and the ASR is a rate under a judge, not a rate of label flips.

7.2 A surrogate loss and the discrete budget problem

The indicator J is not differentiable and is expensive to query, so, as in Section 1, we replace it by a surrogate. The standard choice is the likelihood of an *affirmative prefix* $\mathbf{r}^\star$ such as "Sure, here is…", which models that comply usually begin with. For the suffix threat set,

$$\mathcal{L}(\mathbf{s}) = -\log p(\mathbf{r}^\star \mid q \oplus \mathbf{s}) = \sum_{t=1}^{m^\star} -\log p\big(r_t^\star \mid q \oplus \mathbf{s},\, r_{<t}^\star\big), \tag{12}$$

where the equality is the chain rule of probability. The (budget) problem becomes the discrete problem

$$\min_{\mathbf{s} \in \mathcal{V}^k} \mathcal{L}(\mathbf{s}). \tag{13}$$

It has N^k candidates (for $N \approx 10^5$ and $k = 20$, far too many to enumerate), no continuous structure, and a surrogate gap: a response that begins with the affirmative prefix may still refuse afterwards, and a successful response may not begin with it. This is the same gap discussed in the remark on the two optimization problems, now with a more fragile surrogate.

Linearizing a discrete problem. Let $W_E \in \mathbb{R}^{N \times h}$ be the embedding matrix. The model receives the embedding of token s_i, which is $\mathbf{e}_{s_i}^\top W_E$ for the one-hot vector $\mathbf{e}_{s_i} \in \{0, 1\}^N$. Treat the one-hot rows as real vectors, so that $\mathcal{L}$ becomes a differentiable function of $\mathbf{e}_i \in \mathbb{R}^N$ for each suffix position i, and let $\mathbf{g}_i = \nabla_{\mathbf{e}_i} \mathcal{L} \in \mathbb{R}^N$.

The Taylor expansion of Eq. (2) applies verbatim with $\boldsymbol{\delta} = \mathbf{e}_w - \mathbf{e}_v$, the change caused by replacing token v at position i by token w:

$$\mathcal{L}(\mathbf{s}_{i \leftarrow w}) \;\approx\; \mathcal{L}(\mathbf{s}) + (\mathbf{e}_w - \mathbf{e}_v)^\top \mathbf{g}_i = \mathcal{L}(\mathbf{s}) + g_{i,w} - g_{i,v}.$$

This is the first-order estimate used by HotFlip (Ebrahimi et al., 2018), and it means that all N possible replacements at position i are scored with a single backward pass.

Proposition 7.1 (Best swap under the linear model)**.** *Among all convex combinations λ of one-hot vectors (the simplex), $\min_\lambda \lambda^\top \mathbf{g}_i = \min_w g_{i,w}$, and the minimum is attained at the vertex $\mathbf{e}_{w^\star}$ with $w^\star = \arg\min_w g_{i,w}$. Hence the best replacement at position i under the linear model is $w^\star$, and relaxing the discrete constraint to the simplex loses nothing.*

Proof. For $\lambda \geq 0$ with $\sum_w \lambda_w = 1$, $\lambda^\top \mathbf{g}_i = \sum_w \lambda_w g_{i,w} \geq \sum_w \lambda_w \min_{w'} g_{i,w'} = \min_{w'} g_{i,w'}$, with equality for the vertex at the minimizing coordinate. □

The structure matches the ℓ_1 case of Exercise 1: a linear objective over a set whose extreme points are coordinate vectors is optimized by placing all the mass on one coordinate. The difference from ℓ_∞ FGSM is that there is no sign step spread across coordinates, only a choice of one token.

Why the linear model can only rank. In the classifier case the linearization error shrinks with the step, $O(\|\boldsymbol{\delta}\|^2)$, and BIM exploits this by taking small steps. Here no step is small: $\|\mathbf{e}_w - \mathbf{e}_v\|_2^2 = 2$ for every swap. If the relaxed loss has a β-Lipschitz gradient, the bound derived for FGSM gives $|R| \leq \frac{\beta}{2} \cdot 2 = \beta$ for every swap, a fixed error that does not vanish as the attack proceeds. The estimate therefore cannot be trusted to predict the loss after a swap. It can still be used as a heuristic to *propose* candidates, provided each candidate is checked with the true loss. This is the design principle of the next algorithm. The bound itself assumes smoothness that Transformers do not satisfy globally, so we read it as intuition.

7.3 White-box: Greedy Coordinate Gradient

The Greedy Coordinate Gradient attack (GCG) of Zou et al. (2023) combines the proposal and the check. At each iteration it (i) computes $\mathbf{g}_i$ for all suffix positions with one backward pass, (ii) keeps for each position the K tokens with the most negative gradient entries, (iii) samples B candidate suffixes, each obtained by replacing one random position by one of its top-K tokens, (iv) evaluates the true loss $\mathcal{L}$ of every candidate with forward passes only, and (v) moves to the best candidate.

Algorithm 3 GCG, suffix of length k

Require: request q, target prefix $\mathbf{r}^\star$, suffix length k, iterations T, top-K, batch B

initialize $\mathbf{s} \in \mathcal{V}^k$ (for example, a repeated filler token)
for $t = 1, \ldots, T$ **do**
  $\mathbf{g}_i \leftarrow \nabla_{\mathbf{e}_i} \mathcal{L}(\mathbf{s})$ for $i = 1, \ldots, k$ ▷ one forward and backward pass
  $\mathcal{X}_i \leftarrow$ the K tokens with smallest $g_{i,w}$
  for $b = 1, \ldots, B$ **do**
    draw i uniformly in $\{1, \ldots, k\}$ and w uniformly in $\mathcal{X}_i$; set $\mathbf{s}^{(b)} \leftarrow \mathbf{s}_{i \leftarrow w}$
  end for
  $\mathbf{s} \leftarrow \arg\min_b \mathcal{L}(\mathbf{s}^{(b)})$ ▷ B forward passes
end for
return $\mathbf{s}$

Universal and transferable suffixes. Replacing $\mathcal{L}$ in Eq. (12) by a sum over several requests q_j and several open models m,

$$\mathcal{L}_{\text{ens}}(\mathbf{s}) = \sum_m \sum_j -\log p_m(\mathbf{r}_j^\star \mid q_j \oplus \mathbf{s}),$$

is exactly the loss fusion of Section 6. The optimized suffix is then *universal* (one string works for many requests) and, as in Section 5, an ensemble of surrogates improves transfer to models the attacker cannot differentiate through. Zou et al. (2023) report transfer of such suffixes to closed-source models, but how well it works depends on the target and has changed as models were updated.

Embedding-space attacks. If the attacker has the weights, nothing prevents running PGD directly on the continuous embeddings of the suffix, which is the classifier setting of Section 3.1. This is an upper bound on what token-space attacks can achieve, but it is not a realistic threat model for a user who can only submit text, because an arbitrary embedding vector need not correspond to any token. The result of Carlini et al. (2023) that continuous inputs (images) in multimodal models are much easier to attack than text is the same phenomenon.

7.4 Black-box attacks

The chapter's information axis carries over directly.

Score-based. If the API returns log-probabilities, the surrogate of Eq. (12) can be evaluated without gradients. Random search then does what Square Attack does for images: mutate a few random suffix tokens and keep the change only if the log-probability of the affirmative prefix increases. Andriushchenko et al. (2024) showed that this simple adaptive strategy reaches high success rates against several safety-aligned models. Restricting the logit bias, temperature or returned log-probabilities raises the cost, as for score-based attacks on classifiers.

Judge-based (the decision-based analogue). When only the response text is visible, the only feedback is the verdict of a judge. PAIR (Chao et al., 2023) uses an *attacker LLM* that proposes a rewrite of the request, queries the target, receives the judge's score, and refines the rewrite in the next round. Like the Boundary Attack, it needs no gradients and no scores, and its cost is measured in queries (typically tens to a few hundred for PAIR). The rewrites are readable text, which matters for the defenses below. Multi-turn variants such as Crescendo spread the attack over a conversation of individually benign turns (Vassilev et al., 2025).

Hand-crafted attacks (no optimization). Wei et al. (2023) explain many manual jailbreaks by two failure modes. *Competing objectives*: the model is trained to be helpful and to follow instructions, and the attack makes compliance win (refusal suppression, role-play, forcing an affirmative start). *Mismatched generalization*: pretraining covers a wider distribution $\mathcal{D}_{\text{pre}}$ than safety training $\mathcal{D}_{\text{safe}}$, and an input in $\mathcal{D}_{\text{pre}} \setminus \mathcal{D}_{\text{safe}}$ (Base64, ROT13, rare languages, payload splitting) is understood by the model but not covered by its refusal behavior. Neither mechanism requires a gradient, and the second one has a simple explanation in terms of the coverage of the training distribution, in the spirit of Section 5.

Attack	Information	Problem and surrogate	Solver
GCG	gradients (open weights)	budget (suffix length k), NLL of affirmative prefix	gradient-ranked candidates, forward-pass selection
Embedding-space PGD	gradients (open weights)	budget in embedding space, same NLL	PGD (not a text-only threat)
Random search on log-probs	scores	budget, log-probability of prefix	random token mutation, accept if better
PAIR, TAP-style	labels (judge verdict)	success of a rewrite under a judge	attacker LLM with feedback
Role-play, encodings	none	none (hand-crafted)	human prior about training gaps
Universal suffix, transfer	none (per request)	budget, summed over models and requests	GCG on an ensemble of open models

Table 2: LLM evasion attacks along the axes of Table 1. Entries summarize the cited papers.

7.5 Prompt injection

In a *direct* prompt injection the attacker is the user and tries to override the developer's system prompt. In an *indirect* injection the attacker controls a document d (a web page, an email, a retrieved passage) that the application inserts into the context together with the user's request u and the system prompt $\mathbf{z}$, so the model receives a single sequence $\mathbf{z} \oplus u \oplus d$ (Greshake et al., 2023; Vassilev et al., 2025). The attacker's goal is a behavior b that neither $\mathbf{z}$ nor u asked for (exfiltrating data to a URL, calling a tool, suppressing a source). The threat set is $\{d : \text{the attacker can place } d \text{ in the context}\}$, which is limited by access, not by distance, and the goal can be stated as $\Pr[\text{behavior } b \mid \mathbf{z} \oplus u \oplus d]$ being large. The same optimization techniques apply to d (for instance, optimized "execution triggers" reported by Vassilev et al. (2025)), and the attack can persist through retrieval pipelines.

The root cause is architectural: the model has no reliable mechanism to distinguish instructions from data, because both are tokens in the same sequence. For this reason, defenses that only modify the model reduce but

do not remove the risk, and a common recommendation is to design the application on the assumption that the model may follow injected instructions, by limiting its permissions and the data it can reach (Vassilev et al., 2025).

7.6 Defenses and their limits

- **Perplexity filtering.** GCG suffixes are unnatural token strings with high perplexity under a language model, and a filter that rejects prompts above a threshold detects them (Jain et al., 2023). This defends against that attack and not against PAIR-style rewrites, which are fluent. An adaptive attacker adds a fluency term to Eq. (12), trading attack strength for readability.
- **Randomized perturbation (SmoothLLM).** Robey et al. (2023) query the model on several randomly perturbed copies of the prompt and aggregate the outputs. This is motivated by randomized smoothing (Remark 1.3): optimized suffixes are brittle, and character-level noise breaks them while leaving benign prompts mostly intact. It costs several queries per prompt, and the guarantee is not the certified radius of the classifier setting.
- **Training-time defenses.** Safety training, refusal training and adversarial training on attack prompts raise the cost of attacks, but recent evaluations still find current models vulnerable to direct prompt injection (Vassilev et al., 2025). Adversarial training against GCG-style suffixes faces the same difficulty as in Chapter 1: the inner maximization is expensive and the threat set is open-ended.

Evaluating LLM attacks and defenses

- **The judge is part of the threat model.** The ASR depends on who decides J. Keyword matching on refusals produces both false successes (a hedged answer without a refusal phrase) and false failures. State the judge and validate it on a labeled sample.
- **The surrogate is not the objective.** Low $\mathcal{L}(\mathbf{s})$ means the model starts with the affirmative prefix, not that the full response is harmful. Report success under J, not the loss.
- **Randomness and budget.** Sampling makes success probabilistic (the threshold τ of Definition 7.1). Report the number of samples, the number of queries, and the computational budget of the attack, since a weak attack gives a loose upper bound on robustness exactly as in Remark 1.3.
- **Adaptive attacks.** A defense evaluated only against attacks that were not designed for it (a perplexity filter tested with plain GCG) says little. Follow the checklist of Carlini et al. (2019) and Tramèr et al. (2020): attack the defense, not the base model.

Exercises

1. Verify the equality in Eq. (12) from the chain rule, and explain why $\mathcal{L}$ can be evaluated for all $m^\star$ positions with a single forward pass of the sequence $q \oplus \mathbf{s} \oplus \mathbf{r}^\star$ (teacher forcing).
2. Show that the first-order estimate for a swap uses $g_{i,w} - g_{i,v}$, and explain why the term $g_{i,v}$ can be dropped when ranking replacements w at a fixed position i.
3. In GCG, the cost of one iteration is one backward pass plus B forward passes. Compare this with the cost of one PGD step and of one NES-PGD step on a classifier. When is it worth increasing B rather than K?
4. Design an adaptive attack against a perplexity filter with threshold θ: write the modified objective, and say what happens to the attack success as θ decreases.
5. Classify the following along the axes of Table 1: GCG on an ensemble of open models evaluated on a closed model, random search on log-probabilities, PAIR. For each, say which classifier-setting attack it most resembles and which assumption of that attack fails for LLMs.

8 Hands-On: Implementing Evasion Attacks

The following PyTorch code implements the attacks of this chapter in their simplest form. It assumes a model `model` in `eval()` mode that receives images in $[0, 1]$ (place any normalization inside the model), and batches `x`, `y`. Any CIFAR-10 classifier will do; pretrained robust and non-robust models are available in RobustBench.

8.1 FGSM and PGD

```
import torch
import torch.nn.functional as F

def fgsm(model, x, y, eps):
    x = x.clone().detach().requires_grad_(True)
```

```
    loss = F.cross_entropy(model(x), y)
    grad, = torch.autograd.grad(loss, x)
    return (x + eps * grad.sign()).clamp(0, 1).detach()

def margin_loss(logits, y):
    # max_{j != y} Z_j - Z_y   (positive => misclassified)
    true = logits.gather(1, y[:, None]).squeeze(1)
    other = logits.clone()
    other.scatter_(1, y[:, None], -float("inf"))
    return other.max(1).values - true

def pgd_linf(model, x, y, eps, alpha, steps, restarts=1, loss="ce"):
    best = x.clone()
    fooled = torch.zeros(len(x), dtype=torch.bool, device=x.device)
    for _ in range(restarts):
        delta = torch.empty_like(x).uniform_(-eps, eps)
        xa = (x + delta).clamp(0, 1).detach()
        for _ in range(steps):
            xa.requires_grad_(True)
            logits = model(xa)
            l = F.cross_entropy(logits, y) if loss == "ce" \
                else margin_loss(logits, y).sum()
            grad, = torch.autograd.grad(l, xa)
            xa = xa.detach() + alpha * grad.sign()
            xa = torch.min(torch.max(xa, x - eps), x + eps).clamp(0, 1)
        with torch.no_grad():
            succ = model(xa).argmax(1) != y
        new = succ & ~fooled
        best[new] = xa[new]
        fooled |= succ
    return best, fooled
```

8.2 Carlini–Wagner ℓ_2 (single c, no binary search)

```
def cw_l2(model, x, y, c=1.0, kappa=0.0, steps=200, lr=1e-2):
    # change of variables: x' = (tanh(w) + 1) / 2
    w = torch.atanh((x * 2 - 1).clamp(-1 + 1e-6, 1 - 1e-6)).detach()
    w.requires_grad_(True)
    opt = torch.optim.Adam([w], lr=lr)
    for _ in range(steps):
        xa = (torch.tanh(w) + 1) / 2
        logits = model(xa)
        true = logits.gather(1, y[:, None]).squeeze(1)
        other = logits.clone()
        other.scatter_(1, y[:, None], -float("inf"))
        h = torch.clamp(true - other.max(1).values, min=-kappa)
        dist = ((xa - x) ** 2).flatten(1).sum(1)
        loss = (dist + c * h).sum()
        opt.zero_grad(); loss.backward(); opt.step()
    return ((torch.tanh(w) + 1) / 2).detach()
```

8.3 A score-based attack: NES gradient estimate inside PGD

```
@torch.no_grad()
def nes_grad(model, x, y, sigma=1e-3, n=50):
    g = torch.zeros_like(x)
    for _ in range(n // 2):
        u = torch.randn_like(x)
        lp = F.cross_entropy(model(x + sigma * u), y, reduction="none")
        lm = F.cross_entropy(model(x - sigma * u), y, reduction="none")
        g += (lp - lm).view(-1, *[1] * (x.dim() - 1)) * u
    return g / (n * sigma)   # n queries per estimate

def nes_pgd(model, x, y, eps, alpha, steps, **kw):
    xa = x.clone()
    for _ in range(steps):
        g = nes_grad(model, xa, y, **kw)
        xa = xa + alpha * g.sign()
        xa = torch.min(torch.max(xa, x - eps), x + eps).clamp(0, 1)
    return xa
```

8.4 Transfer from an ensemble of surrogates

```
class LogitEnsemble(torch.nn.Module):
    def __init__(self, models, weights=None):
        super().__init__()
        self.models = torch.nn.ModuleList(models)
        self.w = weights or [1 / len(models)] * len(models)
    def forward(self, x):
        return sum(w * m(x) for w, m in zip(self.w, self.models))

# craft on surrogates, evaluate on the held-out target
surrogate = LogitEnsemble([resnet18, vgg16, densenet121]).eval()
x_adv, _ = pgd_linf(surrogate, x, y, eps=8/255, alpha=2/255, steps=20)
transfer_rate = (target(x_adv).argmax(1) != y).float().mean()
```

8.5 Suggested experiments

1. On a standard (non-robust) CIFAR-10 model, plot accuracy versus $\epsilon \in \{0, 1, 2, 4, 8\}/255$ for FGSM, PGD-20 with cross-entropy and PGD-20 with the margin loss. Repeat on a robust model from RobustBench. Which attack is strongest, and does the ranking change?
2. For PGD, vary α and the number of steps with fixed $\epsilon = 8/255$. Identify a configuration that underestimates attack strength and explain why.
3. Run C&W ℓ_2 with binary search over c on 100 test images. Report the median minimal ℓ_2 distance and compare the fraction below $\epsilon_2 = 0.5$ with PGD-ℓ_2 at that budget.
4. Implement NES-PGD and plot success rate versus number of queries. How many queries are needed to match 90% of white-box PGD success?
5. Craft examples on a single surrogate and on a three-model logit ensemble. Measure transfer to a held-out architecture, with and without momentum (MI-FGSM). Report untargeted and targeted transfer rates.

Implementation pitfalls

- Keep the model in `eval()` mode; batch normalization in training mode changes outputs per batch.
- Put normalization inside the model so that ϵ is expressed in pixel space.
- Evaluate only on initially correctly classified points, or report clean accuracy alongside robust accuracy.
- Check that $\|\mathbf{x}' - \mathbf{x}\|_\infty \le \epsilon$ and $\mathbf{x}' \in [0, 1]^d$ after the attack. Bugs here silently inflate success rates.

Exercises

1. Derive the FGM update for ℓ_1, the case $p = 1$ left open in Proposition 3.1: use the upper bound with $q = \infty$ to find the maximizer. Why is the ℓ_1 steepest ascent step a poor attack in practice?
2. Show that for ℓ_2, rescaling onto the ball and then clipping to $[0, 1]^d$ does not in general give the Euclidean projection onto $\mathcal{B}_2(\mathbf{x}, \epsilon) \cap [0, 1]^d$. (Hint: try $d = 2$, $\mathbf{x} = (0, 0)$, $\epsilon = 1$, $\mathbf{z} = (-1, 2)$.) Why does the argument of Proposition 3.2 not carry over?
3. Explain why cross-entropy may yield vanishing gradients for highly confident predictions, while the margin loss of Eq. (7) does not.
4. Show that Eq. (11) is an unbiased estimator of $\nabla_{\mathbf{x}} \mathbb{E}_{\mathbf{u} \sim \mathcal{N}(0, I)}[\mathcal{L}(\mathbf{x} + \sigma \mathbf{u}, y)]$. (Hint: Stein's lemma.)
5. A defender uses a majority vote over five independently trained networks and reports that PGD on each individual member fails to fool the vote. Describe a correct adaptive attack.
6. Classify each of the following along the three axes of Table 1 (information level, problem, solver): PGD with the margin loss, NES-PGD, HopSkipJump, MI-FGSM on a three-model ensemble. Which pairs differ in exactly one axis?

References

Andriushchenko, M., Croce, F., and Flammarion, N. (2024). Jailbreaking leading safety-aligned LLMs with simple adaptive attacks. *arXiv preprint arXiv:2404.02151*.

Andriushchenko, M., Croce, F., Flammarion, N., and Hein, M. (2020). Square attack: A query-efficient black-box adversarial attack via random search. In *European Conference on Computer Vision (ECCV)*, pages 484–501.

Athalye, A., Engstrom, L., Ilyas, A., and Kwok, K. (2018). Synthesizing robust adversarial examples. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 284–293.

Brendel, W., Rauber, J., and Bethge, M. (2018). Decision-based adversarial attacks: Reliable attacks against black-box machine learning models. In *International Conference on Learning Representations (ICLR)*.

Carlini, N., Athalye, A., Papernot, N., Brendel, W., Rauber, J., Tsipras, D., Goodfellow, I., Madry, A., and Kurakin, A. (2019). On evaluating adversarial robustness. *arXiv preprint arXiv:1902.06705*.

Carlini, N., Nasr, M., Choquette-Choo, C. A., et al. (2023). Are aligned neural networks adversarially aligned? In *Advances in Neural Information Processing Systems (NeurIPS)*.

Carlini, N. and Wagner, D. (2017). Towards evaluating the robustness of neural networks. In *IEEE Symposium on Security and Privacy (S&P)*, pages 39–57.

Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G. J., and Wong, E. (2023). Jailbreaking black box large language models in twenty queries. *arXiv preprint arXiv:2310.08419*.

Chen, J., Jordan, M. I., and Wainwright, M. J. (2020). HopSkipJumpAttack: A query-efficient decision-based attack. In *IEEE Symposium on Security and Privacy (S&P)*, pages 1277–1294.

Chen, P.-Y., Zhang, H., Sharma, Y., Yi, J., and Hsieh, C.-J. (2017). ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 15–26.

Cohen, J., Rosenfeld, E., and Kolter, J. Z. (2019). Certified adversarial robustness via randomized smoothing. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 1310–1320.

Croce, F. and Hein, M. (2020). Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 2206–2216.

Dong, Y., Liao, F., Pang, T., Su, H., Zhu, J., Hu, X., and Li, J. (2018). Boosting adversarial attacks with momentum. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9185–9193.

Ebrahimi, J., Rao, A., Lowd, D., and Dou, D. (2018). HotFlip: White-box adversarial examples for text classification. In *Proceedings of ACL*.

Goodfellow, I. J., Shlens, J., and Szegedy, C. (2015). Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*.

Gowal, S., Dvijotham, K., Stanforth, R., Bunel, R., Qin, C., Uesato, J., Arandjelovic, R., Mann, T., and Kohli, P. (2019). Scalable verified training for provably robust image classification. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4842–4851.

Greshake, K., Abdelnabi, S., Mishra, S., et al. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*.

He, W., Wei, J., Chen, X., Carlini, N., and Song, D. (2017). Adversarial example defenses: Ensembles of weak defenses are not strong. In *11th USENIX Workshop on Offensive Technologies (WOOT)*.

Ilyas, A., Engstrom, L., Athalye, A., and Lin, J. (2018). Black-box adversarial attacks with limited queries and information. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 2137–2146.

Ilyas, A., Santurkar, S., Tsipras, D., Engstrom, L., Tran, B., and Madry, A. (2019). Adversarial examples are not bugs, they are features. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Jain, N., Schwarzschild, A., Wen, Y., et al. (2023). Baseline defenses for adversarial attacks against aligned language models. *arXiv preprint arXiv:2309.00614*.

Katz, G., Barrett, C., Dill, D. L., Julian, K., and Kochenderfer, M. J. (2017). Reluplex: An efficient SMT solver for verifying deep neural networks. In *Computer Aided Verification (CAV)*, pages 97–117.

Kurakin, A., Goodfellow, I., and Bengio, S. (2017). Adversarial examples in the physical world. In *ICLR Workshop Track*.

Liu, Y., Chen, X., Liu, C., and Song, D. (2017). Delving into transferable adversarial examples and black-box attacks. *International Conference on Learning Representations (ICLR)*.

Madry, A., Makelov, A., Schmidt, L., Tsipras, D., and Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)*.

Moosavi-Dezfooli, S.-M., Fawzi, A., and Frossard, P. (2016). DeepFool: A simple and accurate method to fool deep neural networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2574–2582.

Papernot, N., McDaniel, P., Goodfellow, I., Jha, S., Celik, Z. B., and Swami, A. (2017). Practical black-box attacks against machine learning. In *Proceedings of the ACM Asia Conference on Computer and Communications Security (ASIACCS)*, pages 506–519.

Robey, A., Wong, E., Hassani, H., and Pappas, G. J. (2023). SmoothLLM: Defending large language models against jailbreaking attacks. *arXiv preprint arXiv:2310.03684*.

Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., and Fergus, R. (2014). Intriguing properties of neural networks. In *International Conference on Learning Representations (ICLR)*.

Tramèr, F., Carlini, N., Brendel, W., and Madry, A. (2020). On adaptive attacks to adversarial example defenses. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tramèr, F., Kurakin, A., Papernot, N., Goodfellow, I., Boneh, D., and McDaniel, P. (2018). Ensemble adversarial training: Attacks and defenses. In *International Conference on Learning Representations (ICLR)*.

Tramèr, F., Papernot, N., Goodfellow, I., Boneh, D., and McDaniel, P. (2017). The space of transferable adversarial examples. *arXiv preprint arXiv:1704.03453*.

Uesato, J., O'Donoghue, B., van den Oord, A., and Kohli, P. (2018). Adversarial risk and the dangers of evaluating against weak attacks. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 5025–5034.

Vassilev, A., Oprea, A., Fordyce, A., Anderson, H., Davies, X., and Hamin, M. (2025). Adversarial machine learning: A taxonomy and terminology of attacks and mitigations. Technical Report NIST AI 100-2e2025, National Institute of Standards and Technology.

Wei, A., Haghtalab, N., and Steinhardt, J. (2023). Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems (NeurIPS)*.

Wong, E. and Kolter, J. Z. (2018). Provable defenses against adversarial examples via the convex outer adversarial polytope. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 5286–5295.

Xie, C., Zhang, Z., Zhou, Y., Bai, S., Wang, J., Ren, Z., and Yuille, A. L. (2019). Improving transferability of adversarial examples with input diversity. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2730–2739.

Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., and Fredrikson, M. (2023). Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*.